

OS Structures & Linux FS

Study Guide covering Lecture 2 and Section 2.

PART 1: OS STRUCTURES (LECTURE 2)

1. Operating System Services

Operating systems provide an environment for the execution of programs. These services are divided into two main categories:

A. Services that are helpful to the USER:

- **User Interface (UI):** Command-Line (CLI), Graphics User Interface (GUI), Touch-screen, or Batch.
- **Program Execution:** Loading a program into memory, running it, and ending execution.
- **I/O Operations:** Reading from or writing to files or devices.
- **File-system Manipulation:** Creating, deleting, searching, and managing permissions of files/directories.
- **Communications:** Exchange of info between processes (via shared memory or message passing).
- **Error Detection:** Constantly monitoring CPU, memory, and I/O to ensure correct computing.

B. Services for efficient operation of the SYSTEM itself:

- **Resource Allocation:** Allocating CPU cycles, memory, and I/O to multiple users/jobs concurrently.
- **Logging (Accounting):** Keeping track of which users use how much of the computer resources.
- **Protection and Security:** Ensuring access to resources is controlled and defending against outsiders.

💡 توضيح بالعربي: ليه الـ OS بيقدم نوعين من الخدمات؟

الـ OS بيشتغل خدام عندك (User) وخدام عند نفسه (System).

عشانك: بيوفرلك واجهة سهلة (GUI)، بيشتغل البرامج، بيفتحلك الملفات، وبيطلعلك رسالة error لو حاجة باظت.

عشان نفسه (ولضمان كفاءة الجهاز): بيقسم الرامات بين البرامج بالعدل (Resource Allocation). بيسجل مين استهلك إيه (Logging)، ويحمي ملفاتك من إن برنامج تاني أو يوزر تاني ييوظها (Protection).

2. System Calls & APIs

System Call: The programming interface to the services provided by the OS. Typically written in C or C++.

What is an API?

Programs rarely use system calls directly. Instead, they use an **Application Programming Interface (API)**. The API hides the complex details of the OS interface from the programmer.

Why use APIs rather than system calls?

- **Portability:** A program using an API can compile and run on any system that supports the same API (e.g., standard C library works on Linux and Windows).
- **Ease of Use:** Actual system calls are often detailed and difficult to work with. APIs provide simpler functions.

- **Win32 API:** for Windows.
- **POSIX API:** for UNIX, Linux, Mac OS X.
- **Java API:** for the Java Virtual Machine (JVM).

Types of System Calls

System calls can be grouped roughly into six major categories:

1. Process Control: create/terminate process, wait, allocate/free memory.

2. File Management: create/delete file, read, write, open, close.

3. Device Management: request/release device, read, write.

4. Info Maintenance: get/set time or date, get/set system data.

5. Communications: create/delete connection, send/receive messages.

6. Protection: get/set file permissions.

System Call Implementation

- Each system call is associated with a specific **number**.
- The OS maintains a **table** indexed by these numbers.

- When an API makes a request, the system call interface invokes the intended system call in the OS kernel and returns the status/values.

💡 توضيح بالعربي: قصة الـ System Call والـ API

تخيل إنك في مطعم (إنت البرنامج). الـ **Kernel (النواة)** هو الشيف اللي في المطبخ اللي بيعمل الأكل الفعلي. الـ **System Call** هو المطبخ نفسه وكيفية الطبخ. إنت كزبون مش مسموحدك تدخل المطبخ وتطبخ بنفسك. الـ **API** هو "الجرسون". إنت بتطلب من الجرسون أورد زى "عايز أطبع ورقة" `printf()`، الجرسون بياخد طلبك، يدخل للـ OS، ينفذ الـ System Call المعقد اللي اسمه `write()`، ويرجعك بالنتيجة. الـ API بيخلي الكود بتاعك يشتغل على أي جهاز (Portability) ومريح للمبرمج.

3. Linkers & Loaders

How does a source code file become a running process?

1. **Compiler:** Converts source code (e.g., `main.c`) into a *relocatable object file* (e.g., `main.o`).
2. **Linker:** Combines multiple object files and standard libraries into a single *binary executable file* (e.g., `a.out` or `.exe`).
3. **Loader:** When you run the program, the loader brings the executable file from the disk into main memory (RAM) to be executed.

Dynamically Linked Libraries (DLLs): Modern systems don't link all libraries into every executable (which would waste space). Instead, libraries are loaded into memory *once* and shared by all programs that need them.

Why Applications are OS Specific?

You can't run a Windows `.exe` on a Mac or Linux natively because:

- Each OS provides its own unique system calls.
- Each OS has its own file formats for executables.

Workarounds: Use interpreted languages (Python), Virtual Machines (Java), or recompile the C code on the target OS.

💡 توضيح بالعربي: رحلة الكود من الكتابة للتشغيل

1. **Compiler:** بيترجم الكود اللي إنت كاتبه للغة الآلة (Object file).
2. **Linker:** لو إنت مستخدم أكواد جاهزة (Libraries) زي دالة الطباعة، الـ Linker يربط ملفك بالملفات دي عشان يطلعك ملف واحد نهائي جاهز للتشغيل (Executable).

3. **Loader**: ده اللي بياخد الملف النهائي من الهارديسك، يحطه في الرامات، ويبدأ يشغله.

ليه ال DLLs مهمة؟ بدل ما كل برنامج ينسخ جواه كود الطباعة وياخد مساحة كبيرة، ال OS بيحط كود الطباعة مرة واحدة بس في الرامات، وأي برنامج يعوزه يـ (Link) معاه وهو شغال (Dynamically).

4. OS Design, Implementation & System Boot

OS Design Goals

Designing an OS is highly creative and involves conflicting goals:

- **User Goals:** OS should be convenient, easy to use, reliable, safe, and fast.
- **System Goals:** OS should be easy to design, implement, maintain, error-free, and efficient.

Policy vs. Mechanism

An important principle in OS design is to separate these two concepts:

- **Policy:** *What* will be done? (e.g., Which user gets more CPU time?)
- **Mechanism:** *How* to do it? (e.g., The timer and context switch that actually changes the CPU from one user to another).

Separating them allows maximum flexibility. If you want to change the policy later, you don't have to rewrite the mechanism.

System Boot Process

1. When power is initialized, execution starts at a fixed memory location (ROM or EEPROM).
2. A small piece of code called the **Bootstrap loader** (or BIOS/UEFI) runs.
3. Its job is to locate the **Kernel** on the disk, load it into memory (RAM), and start it.
4. Modern systems use a two-step process: BIOS loads a bootloader (like **GRUB**), which then allows you to choose and load the OS kernel.

توضيح بالعربي: Policy vs Mechanism

ال **Policy (السياسة/القرار)**: بتحدد "ليه اللي المفروض يحصل". مثلاً: قرار إن الموظفين يمشوا الساعة 5.
ال **Mechanism (الآلية)**: بتحدد "إزاي ده يحصل". مثلاً: جهاز البصمة اللي بيقتل الباب الساعة 5.

فصلهم عن بعض مهم في برمجة الـ OS عشان لو حيينا نغير السياسة ونخلي الموظفين يمشوا الساعة 6 (تغيير الـ Policy)، مش هحتاج نغير جهاز البصمة نفسه (الـ Mechanism)، هنغير الإعدادات بس!

PART 2: LINUX DEEP DIVE (SECTION 2)

5. Users and Accounts in Linux

What is a user account?

A systematic approach to track and monitor the usage of system resources. An account has: Username, UID (User ID), GID (Group ID), Home directory, and a Default shell.

Inside `/etc/passwd`, an account looks like this:

```
john:x:1002:1002:,,:/home/john:/bin/bash
```

<code>john</code> = Username	<code>x</code> = Password (encrypted in <code>/etc/shadow</code>)	<code>1002</code> = User ID (UID)	<code>1002</code> = Group ID (GID)
<code>,,</code> = User Info (empty here)	<code>/home/john</code> = Home Directory	<code>/bin/bash</code> = Default Shell	

1. Root User

Main account created during installation. Highest privilege (admin). **UID = 0.**

2. Regular User

Normal account. Can only access authorized files/services. **UID >= 1000.**

3. Service Account

Created by packages when installed. Dummy user, no home directory. **0 < UID < 1000.**

User Management Commands

<code>id username</code>	Display user ID.
--------------------------	------------------

<code>tail /etc/passwd</code>	Get info about users in the system.
-------------------------------	-------------------------------------

<code>tail /etc/shadow</code>	Get users' passwords (encrypted).
-------------------------------	-----------------------------------

<code>sudo su -</code>	Switch to the root user.
------------------------	--------------------------

```
su username
```

Switch to a specific user.

```
useradd -u [id] -d [dir] -s [shell] username
```

Add a new user with specific ID, home dir, and shell. Alternatively, use `adduser username` for an interactive prompt.

```
passwd username
```

Add or change password for a user.

```
userdel -r username
```

Delete user and their home directory.

6. Linux File System Hierarchy

Unlike Windows which uses C:\ and D:\ drives, Linux puts everything under a single "root" directory represented by a forward slash `/`.

/ - The Root directory. Top level.

/etc - Configuration files (passwd, shadow).

/var - Variable data (Databases, log files, emails).

/bin - User commands / binaries (like ls, cat).

/sbin - System administration commands.

/lib - Kernel modules & shared C libraries.

/home - Standard user home folders (/home/ali).

/root - Super user (admin) home folder.

/tmp - Temporary files (auto-deleted).

/dev - System devices (Hard disks, USBs).

/boot - OS Boot files (GRUB).

/usr - Shared files/programs between users.

توضيح بالعربي: شجرة ملفات لينكس 💡

لينكس مفيش فيه ال Partitions زي ال Windows (مفيش C و D). كل حاجة بتبدأ من جذر واحد اسمه ال

./ Root

عشان تفهمهم بسرعة: أي إعدادات للنظام بتلاقيها في /etc, أي حاجة حجمها بيتغير زي اللوجات في /var, ملفات اليوزرز العاديين (الديسكوب والداونلودز بتاعتهم) في /home, والأوامر اللي بنكتبها في ال Terminal بتكون متخزنة كبرامج في /bin و /sbin.

7. Absolute vs. Relative Paths

Absolute Path

Specifies the location of a file/directory starting from the root directory /.

Example: /home/jono/photos

Relative Path

Specifies the location relative to your present working directory (pwd). Never starts with a /.

Example: ./photos or ../../usr/lib

💡 توضيح بالعربي: الفرق بين المسار المطلق والنسبي

Absolute (المطلق): ده العنوان الكامل من أول باب العمارة (ال Root /). مهما كنت إنت واقف فين في النظام، المسار ده هيوصلك صح. (مثال: الشارع الرئيسي، عمارة 5، الدور الثاني، شقة 3).

Relative (النسبي): ده العنوان بالنسبة لمكانك الحالي. بيبدأ ب . (المكان اللي إنت فيه) أو . . (ارجع خطوة لورا). مفيد لو إنت قريب من الملف بدل ما تكتب العنوان كله من الأول. (مثال: اخرج من الأوضة دي وادخل الأوضة اللي جنبها).

8. File & Directory Commands

Navigating the File System

pwd

Print working directory.

cd path

Change directory.

```
cd -  
Go to last working directory.
```

```
cd ~ or cd  
Go to home directory.
```

```
cd ..  
Go up to parent directory.
```

```
cd .  
Current directory (does nothing).
```

Creating, Moving, Copying, Deleting

```
touch file1 file2
```

 Create empty files.

```
mkdir dir1 dir2
```

 Create directories.

```
cp source dest
```

 Copy a file.

```
cp -r source_dir dest_dir
```

 Copy a directory and its contents recursively.

```
mv source dest
```

 Move or rename files/directories. Use `-i` to prevent accidental overwrite.

```
rm file1
```

 Delete a file.

```
rmdir dir1
```

 Delete an empty directory.

```
rm -r dir1
```

 Delete directory and contents (prompts for confirmation).

```
rm -rf dir1
```

 Delete directory and contents forcefully (NO questions asked. Dangerous!).

Viewing and Editing File Content

```
cat fname
```

 Print whole file content.

```
more fname
```

 View content page by page (Space: next page, q: quit).


```
head -n 5 fname
```

Show the first 5 lines.

```
tail -n 7 fname
```

Show the last 7 lines.

```
gedit fname
```

Graphical text editor.

```
nano fname
```

Simple command-line text editor.